AIGC 5500 | WEEK 2 | 4% OF FINAL GRADE

Lab 02: Making Training Work

Due date posted on Blackboard

You are given a network that will not train. Diagnose why, then fix it through a sequence of documented changes — applied and measured one at a time, not all at once. What is being assessed is whether you can tell which of initialization, normalization, or the optimizer is actually responsible for a training failure, rather than changing everything and hoping.

Starter code

You are given lab02_starter.py, containing BrokenNet and a small toy classification task. Do not edit this file — copy what you need from it into your own work. Do not change the architecture (depth, width, or activation function); every fix in this lab is a change to initialization, normalization, or training, not to what the network is built from.

```
class BrokenNet(nn.Module):
    def __init__(self, depth=8, width=32, in_features=20):
        ...
        for layer in self.hidden:
            nn.init.normal_(layer.weight, mean=0.0, std=0.3)
            nn.init.constant_(layer.bias, -2.0)

    def forward(self, x):
        for layer in self.hidden:
            x = torch.relu(layer(x))
        return self.out(x)
```

Run lab02_starter.py once, unmodified, before doing anything else. It trains BrokenNet for 50 epochs with plain SGD and prints the loss. Confirm for yourself that the loss barely moves from its starting value — this is your baseline, and every later stage in your report is measured against it.

What to build

Three pieces, in order.

- **Diagnosis.** Before changing anything, determine why BrokenNet does not train. At minimum, check the per-layer gradient magnitude after one backward pass (the same check from the Week 2 module) and state, in writing, which failure mode it points to.
- **A sequence of fixes, applied one at a time.** Starting from BrokenNet, apply changes one at a time — retraining and recording a loss curve after every single change, not after the final combination. Your sequence must include a change to initialization and a change to normalization. You may also try a different optimizer or a learning rate schedule, but you are not guaranteed either will help; that is for you to find out, not assume.
- **A conclusion.** State which single change contributed the most, and explain why using the mechanism behind it — not "it worked better," but what that change actually did to the gradient or the activations that the others did not.

Requirements

Your submission must do all of the following. Each is checked directly during marking.

- Reproduce the failure first, and report a concrete diagnostic (a gradient magnitude, not just "the loss did not move") that identifies which failure mode is present.
- Apply your fixes in stages. Each stage changes exactly one thing from the previous stage and reports its own loss curve.
- Include a stage that fixes initialization and a stage that fixes normalization, each in isolation before any combination.

- If you try a different optimizer or a schedule, report its effect honestly, including if it does not help — a stage that made no difference is still a result, and explaining why is worth more than omitting it.
- Plot all stages' loss curves together (or a clear table of final losses per stage) so the marginal contribution of each change is visible at a glance.
- State your final diagnosis and which change mattered most, with a mechanism-level explanation.

Watch out

A zero gradient cannot be fixed by changing the optimizer. If your diagnosis finds the gradient is exactly zero at some layer, no learning rate, momentum, or adaptive optimizer will revive it, because every optimizer update is a function of the gradient, and there is nothing there to act on. If your optimizer-only stage does nothing, that is not a failed experiment — it is evidence about where the real problem lives, and belongs in your conclusion.

Repository requirements

The repository is as much a part of the submission as the code. It must contain the following, and it must not contain your virtual environment.

Item	Requirement
.gitignore	Present, committed before your first code commit, and excluding .venv, __pycache__ and .env
README.md	Explains what was wrong with BrokenNet, what you changed at each stage, and how to reproduce your results
requirements.txt	Generated with pip freeze, including torch
Your code	Your diagnosis, each fix stage, and the final comparison, as scripts or a single notebook
Commit history	At least three commits, each a coherent change, with messages that describe what changed

README template

You may use this structure directly. Replace the content, not the shape.

```
# Making Training Work

Diagnosing and fixing a network that would not train, one change at a
time.

## Setup
python -m venv .venv
source .venv/bin/activate  # Windows: .venv\Scripts\activate
pip install -r requirements.txt

## Run
python diagnose.py    # reproduces the failure and reports the gradient check
python fix_stages.py  # runs each fix stage and saves a comparison plot

## Diagnosis
A short paragraph: what the gradient check showed, and which failure
```

```
mode it pointed to.

## Stages and results
Stage 0 (baseline):            final loss 0.689
Stage 1 (+ initialization):    final loss 0.XXX
Stage 2 (+ normalization):     final loss 0.XXX
Stage 3 (+ optimizer/schedule, optional): final loss 0.XXX

## Conclusion
Which stage mattered most, and why, in terms of what it changed about
the gradient or the activations.

## Known limitations
Anything you are aware of that does not work, or that you would
improve with more time.
```

Why one change at a time

Applying every fix at once and watching the loss drop tells you the combination worked; it tells you nothing about which piece of it mattered. This is the same principle behind Lab 01's comparison and every ablation you will run for the rest of this course: a result is only interpretable against a controlled baseline that differs from it in exactly one place. A four-line diff between two working notebooks is a weaker submission than a five-stage sequence that shows the same ending point arrived at deliberately.

How to submit

Post the URL of your GitHub repository to the Lab 02 submission on Blackboard. Do not upload files, and do not submit a link to a single file within the repository.

```
https://github.com/your-username/your-repository
```

Make sure the repository is public. If you prefer to keep it private, you must share it with the instructor account below. Sharing with any other account does not count as a submission and will be rejected.

Instructor GitHub account

`hossein.pourmodheji@humber.ca`

Add this account as a collaborator under Settings, Collaborators in your repository. Any other account will be rejected.

A repository that a marker cannot open is treated as not submitted, and the late policy applies from the point at which access is granted.

Before you submit, run this check

Clone your own repository into a different folder, create a fresh virtual environment, install from requirements.txt, and reproduce every stage following only your README. If every stage runs cleanly and agrees with what you reported, your submission is complete. If it does not, you have found the problem before your marker did.